Операционные системы и системное программирование

Практические и лабораторные занятия

Преподаватель: Поденок Леонид Петрович

к. 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

Версия от 2026.01.13

prep@newman.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2026

Оглавление

Общие замечания....3
Замечания относительно среды выполнения лабораторных и курсовых....8
Практические занятия....9
Лабораторные....10
Лабораторная работа No 1. Знакомство с Linux/Unix и средой программирования. POSIX-совместимая файловая система....10
Лабораторная работа No 2. Понятие процессов....12
Лабораторная работа No 3. Взаимодействие и синхронизация процессов....14
Лабораторная работа No 4. Задача производители-потребители для процессов....15
Лабораторная работа No 5. Потоки исполнения, взаимодействие и синхронизация....16
Лабораторная работа No 6. Работа с файлами, отображенными в память....17
Лабораторная работа No 7. Блокировки чтения/записи....19
Лабораторная работа No 8. Сокеты. Взаимодействие процессов....20
Приложение А. Структура проекта и примеры makefile....22
Один исходный файл на C....22
Библиотека и программа, ее использующая....22
Один исходный файл на ассемблере....24
Приложение Б. Требования к оформлению исходных файлов....25
Приложение В. Требования к оформлению лабораторной работы....27
Приложение Г. Требования к почтовому сообщению....28

Общие замечания

!!! ВАЖНО !!! Порядок защиты лабораторных работ

Практика показывает, что треть студентов затягивает сдачу лабораторных работ и подходит к сессии с долгами.

Исходя из задачи допуска к сессии максимального количества студентов, в первую очередь к защите допускаются студенты без и с минимальным количеством отставаний/долгов.

Например, в середине семестра Иванов и Петров отстают на две лабораторные или в конце сессии имеют в долгах, Сидоров – одну, а Кузнецов собирается сдавать очередную работу. К защите первым допускается Кузнецов, за ним Сидоров сдает долг, после чего Иванов и Петров сдают по одной лабораторной из долга. Таким образом к сессии допускается больше студентов.

Аналогичная ситуация в случае отсутствия долгов — кто впереди, сдает первым, кто отстает — последним. В любом случае те, кто должен больше, допускаются в последнюю очередь. Соответственно, студент может спрогнозировать свои шансы быть допущенным к сессии и не мешать успевающим быть допущенными к сессии.

Общие требования

В разделе представлены общие требования к лабораторным работам, курсовому проекту (далее работа), к организации структуры каталогов с файлами лабораторной работы и их оформлению для предоставления преподавателю на проверку или защиту.

Цель — обеспечить единообразность выполнения и оформления лабораторных и курсовых работ по дисциплине.

Предоставление работы на проверку и защиту может осуществляться как посредством оправки почтового сообщения, содержащего архив каталога с работой, либо посредством передачи материалов работы любым другим способом по выбору преподавателя.

Состав и формат передаваемого материала одинаков для всех лабораторных работ и и описан ниже. Требования к передаче работы преподавателю посредством почтового сообщения описаны в Приложении Г.

Размещение лабораторных и курсовых работ на ресурсах вне кафедры запрещается.

1) Файловая система ОС, в среде которой выполняются практические занятия, лабораторные и курсовые работы, должна поддерживать жесткие и символические ссылки, не иметь ограничений на алфавит имен файлов, поддерживать три даты и быть полностью совместимой с POSIX[1] на уровне доступных непривилегированному пользователю системных вызовов.

2) Разработка, компиляция и сборка проекта работы, отладка и тестирование, полностью ведется в POSIX-совместимой ОС с использованием компилятора `gcc` и отладчика `gdb`.

3) Сборка проекта при защите должна выполняться с помощью утилиты `make` (`gmake`). Студент должен понимать содержимое файла `Makefile` и уметь объяснить.

Допускается использовать на стадии разработки и отладки использовать системы сборки, предоставляемые IDE, однако все следы их работы перед архивированием проекта для передачи преподавателю должны быть удалены. Это также касается временных и «служебных» файлов, помещаемых в каталоги некоторыми IDE, ОС (MacOS) и программами индексирования. Достигается запуском `tar` с опциями, ограничивающими включение расширенных атрибутов (см. `tar(1)`). Если IDE не формирует `Makefile`, это должен сделать студент руками.

В составе `Makefile` проекта не должно быть целей, связанных с запуском собранных программ — все программы запускаются непосредственно из командной строки. Допускается использование специально созданного shell-скрипта[2] для запуска программы с большим количеством параметров, но опять же, из командной строки.

Допускается использовать систему сборки `cmake` с генератором «Unix Makefiles» и с опцией `--verbose`.

1 https://pubs.opengroup.org/onlinepubs/9799919799/

2 Все скрипты в проекте — shell-скрипты.

4) Язык программирования для выполнения лабораторных работ — C в версии 2011 года (ISO/IEC 9899-2011) и выше (`-std={c11|c17|c20|c23} -pedantic`).

5) Проект должен компилироваться и собираться компилятором `gcc` без каких либо серьезных предупреждений. Допускается подавление незначимых предупреждений, типа неиспользуемых переменных и параметров функций (`-Wno-unused-*`), которые не влияют на работу программы. Все остальные предупреждения должны быть устранены.

Преподаватель должен контролировать процесс сборки и в случае появления предупреждений отклонить работу без объяснений, как устранить проблему — студент должен в этом разобраться самостоятельно, поскольку сдал экзамен по соответствующему курсу.

6) Все программы, разрабатываемые в рамках курса, прежде чем они будут представлены преподавателю на проверку и защиту, должны подлежать надлежащей отладке и тестированию.

Все программы, в которых есть выделение памяти из кучи, должны быть проверены с помощью `valgrind(1)` или с помощью других ПС аналогичной функциональности на использование неинициализированных данных, а также на ошибки, связанные с выделением и освобождением памяти (`malloc()/free()`). На защите лабораторной программы должны запускаться под `valgrind(1)` со следующими опциями и файлом для вывода сообщений `valgrind`:

```
$ valgrind --leak_check=full --show-leak-kinds=all -s -q lab_prog 2> err
```

Файл `err`, куда `valgrind` посылает свои сообщения, должен быть пустым.

7) Обязательные опции `gcc`:

в режиме отладки (debug)

```
-g -ggdb -std=c11 -pedantic -W -Wall -Wextra
```

в режиме выпуска (release)

```
-std=c11 -pedantic -W -Wall -Wextra -Werror
```

8) Проекты лабораторных работ и курсовой по ОСиСП должны располагаться в отдельных каталогах следующей структуры:

```
$ tree -L 2
tar_working_dir     -- каталог для архивирования лабораторных и курсовой
└── Иванов␣И.О.
    ├── course-work -- каталог для курсовой работы/проекта
    ├── lab01
    ├── lab02
    ├── lab03
    ├── lab04
    ├── lab05.1
    ├── lab05.2
    ├── lab06
    ├── lab07
    └── lab08
```

Здесь «`Иванов␣И.О.`» — фамилия и инициалы студента[3]. Внутри каталога с именем студента располагаются подкаталоги с лабораторными и курсовым проектом.

9) Файлы в составе проекта должны контролироваться git[4] — каждая лабораторная и курсовой проект должны иметь свой отдельный локальный git-репозиторий.

3 Глиф '␣' (U+2423) в компьютерной литературе используется для указания необходимого пробела (0x20).

4 Читаем книгу Чакона и Страуба «Про Гит» (лежит на ftp://lsi.bas-net.by)

Использование github и прочих нелокальных репозиториев для разработки курсового проекта и лабораторных работ запрещается. Размещение лабораторных и курсовых работ на публичных ресурсах вне кафедры запрещается.

Коммиты в репозитории должны, как минимум, чисто собираться. Предоставляемая на проверку/защиту версия должна иметь отдельный тег.

Перед подготовкой проекта к проверке/защите студенту следует убедиться, что все изменения зарегистрированы в репозитории, а все промежуточные файлы и результаты сборки отсутствуют[5]. Файлы, которые возникают в процессе сборки или запуска программы, не должны контролироваться git — они указываются в файле `.gitignore`.

```
$ cd lab06
$ make clean
$ git status
On branch master
Your branch is up to date with 'origin/master'.
nothing to commit, working tree clean
```

10) Каталог с лабораторной работой должен содержать:

- исходные файлы на языке C с комментариями на русском языке[6] без использования сленга и в кодировке utf8 (требования к оформлению приведены в «Приложение Б. Требования к оформлению исходных файлов»);

- `Makefile` — файл управления сборкой проекта[7] (`CmakeLists.txt`, если используется система сборки `cmake`, при этом запуск `cmake` выполняется в отдельном каталоге `build/`);

- файл `README` с кратким описание проекта, его сборки, запуска и тестирования в текстовом формате на русском языке в кодировке utf8;

- скрипты, входные или иные данные, если они необходимы для демонстрации и тестирования программ проекта;

- отчет о лабораторной работе в формате `.pdf` (Требования к оформлению приведены в Приложении В).

Ниже представлен пример содержимого каталога с проектом лабораторной работы № 6:

```
$ tree -a lab06
lab06
├── .git
⋮
├── .gitignore
├── Makefile           -- файл, управляющий сборкой
├── build              -- каталог с результатами сборки
│   ├── debug          -- каталог с результатами отладочной сборки
│   │   ├── .gitignore -- присутствует чтобы git отслеживал пустой каталог
│   │   ├── generator  -- программа генерации индекса
│   │   ├── sort_index -- программа сортировки индекса
│   │   └── viewer     -- программа просмотра индекса
│   └── release        -- каталог с результатами выпускной сборки
│       └── .gitignore -- присутствует чтобы git отслеживал пустой каталог
├── lab06.pdf          -- отчет о лабораторной работе
├── readme.txt         -- краткое описание процесса сборки и тестирования
├── src                -- каталог с исходными файлами и заголовками проекта
│   ├── generator.c
│   ├── sort_index.c
│   ├── utility.h
│   └── viewer.c
```

5 Вывод git status должен быть чист (.gitignore).

6 Допускается комментировать код на чистом и грамотном английском языке.

7 Читаем для начала соответствующую статью на https://www.opennet.ru/docs/RUS/make_compile/

```
└── test -- каталог для тестирования содержит симлинки на собранные прогр.
    ├── generator -> ../build/debug/generator
    ├── index_db
    ├── index_db.sorted
    ├── sort_index -> ../build/debug/sort_index
    └── viewer -> ../build/debug/viewer
```

Вышеприведенная структура каталогов для других лабораторных не является обязательной — допускается все вышеуказанные файлы держать в одном каталоге, что, с одной стороны, упрощает написание `Makefile`, но, с другой, превращает каталог с проектом в свалку.

Все файлы, сгенерированные в процессе сборки, не должны контролироваться git. Для этой цели они вносятся в файл `.gitignore`. Для очистки каталога от этих файлов можно в `Makefile` указать .phony-цель для очистки каталога проекта. Также можно создать отдельный скрипт, удаляющий все, что упомянуто в `.gitignore`.

11) Все файлы в каталогах должны иметь правильные атрибуты. Наличие атрибута исполняемости у файлов, не являющимися исполняемыми (`file(3)`), расценивается как непонимание сути курса, такая лабораторная не принимается к защите. Все текстовые файлы должны иметь концы строк UNIX (`\n => LF`).

12) Файл управления сборкой проекта должен содержать следующие цели:

`clean` – очистка каталога с проектом от результатов сборки и временных файлов, создаваемых IDE и программами индексации (допускается внесение таких файлов в `.gitignore` с последующим удалением упомянутых в `.gitignore` файлов);

`all` – сборка проекта целиком (пересборка).

Допускается использовать скрипты, реализующие очистку и полную пересборку.

13) Файл управления сборкой проекта должен выполнять сборку в режиме отладки и режиме выпуска:

```
$ make MODE=debug
$ make MODE=release
```

Пример такого `makefile` приведен в «Приложение А. Структура проекта и примеры makefile ».

14) С целью предварительной проверки лабораторной перед ее защитой студент предоставляет архивированный каталог с проектом преподавателю. Преподаватель или студент в присутствии преподавателя разархивирует каталог с проектом работы, выполняет сборку, запуск и демонстрацию в строгом соответствии с файлом, содержащим краткое описание проекта, порядок сборки, запуска и демонстрации/тестирования (см. п. 10). Все операции выполняются из оболочки (виртуального терминала). Настоятельно рекомендуется установить и изучить файловый менеджер `mc`[8], который существенно облегчает эффективную навигацию по файловой системе и операции с файлами, не ограничивая возможности работать прямо в оболочке.

Использование для операций с файлами оконных приложений и рабочего стола не допускается. Все манипуляции с файлами производятся либо прямо в оболочке, либо в двухпанельном файловом менеджере Midnight Commander. В случаях ~~саботажа~~ непонимания вышеизложенного преподавателю следует попросить студента открыть вторую консоль (`Ctrl-Alt-Fn`) и выполнить все манипуляции со сборкой и запуском программы из нее.

Для предоставления преподавателю каталог проекта архивируется снаружи структуры каталогов лабораторных работ архиватором `tar` (см. п.8). Для сокращения размеров архива можно использовать сжатие в форматы, которые поддерживаются архиватором `tar`, например `gzip`, `xz`. Архивация и сжатие выполняется из оболочки. Использование оконных приложений для этих целей не допускается[9].

8 Midnight Commander

9 Поскольку такими «архиваторами» некорректно обрабатываются текущий каталог и даты, в результате чего при

Использование архиваторов rar, zip и других, которые не сохраняют атрибуты, либо не являются фильтрами[10], запрещается.

Каждая лабораторная и/или курсовой проект должны быть архивироваться в отдельный файл. Создание архивов, содержащих несколько лабораторных, не допускается.

Создание архива с проектом и вообще всю работу с `tar` следует выполнять из оболочки, поскольку оконные обертки вместо явного перехода в каталог «`tar_working_dir`», из которого должен запускаться `tar`, могут работать из другого и использовать возможность установки корневого каталога архива в значение «`tar_working_dir`» и делают это неправильно. В результате дата каталога «`tar_working_dir`», а в ряде случаев и его подкаталогов, при разворачивании устанавливается в текущую, что является бессмыслицей — файл из архива не может быть моложе архива. Некоторые реализации `tar` в таком случае могут отказаться разворачивать архив.

Особое внимание на детали работы `tar` следует обратить в ОС MacOS и найти «правильные» опции. Команда для сборки архива на примере лабораторной № 6:

```
$ cd tar_working_dir    --  переходим в каталог работ по ОсиСП
$ ls t -1               --  посмотрим, что там есть?
Иванов И.О.
$ tar -zcf Иванов_И.О.lab06.tar.gz  "Иванов И.О./lab06"
$ ls -1
Иванов И.О.
Иванов_И.О.lab06.tar.gz -- этот файл имеет правильную структуру каталогов
```

15) Все архивы одной и той же лабораторной или курсового проекта должны иметь одно и тоже имя в течение всего курса. Имя архива должно начинаться с фамилии и инициалов студента и содержать после них номер лабораторной. Формат имени архива должен оставаться постоянным в течение всего курса, т.е в имени архива с лабораторной может изменяться только номер лабораторной.

Например, архив с лабораторной № 2 будет иметь имя «`Иванов_И.О.lab02.tar.gz`».

Имя архива не должно иметь пробелов — вместо пробелов используем подчеркивания.

16) Наличие в архиве иных файлов, в том числе размещаемых операционной системой или IDE для целей индексации и прочих, не допускается. Если такие файлы вдруг попали в архив, они должны быть удалены из архива, прежде чем будут отправлены на проверку. Это достигается архивированием каталога с проектом без сжатия, после чего «лишние» файлы удаляются из архива.

Обычно имена таких файлов начинаются с точки и в используемых по умолчанию режимах отображения в файловых менеджерах не появляются в выдаче. Следует включить режим отображения всех файлов в каталогах.

После очистки архива от мусора его можно сжать и отправить/предоставить на проверку. Для справки, команды архивации, очистки и сжатия:

```
$ tar -cf Иванов_И.О.lab06.tar" "Иванов И.О./lab06"         # архивирование
$ tar --delete -f Иванов_И.О.lab06.tar "Иванов И.О./.мусор" # удаление
$ gzip "Иванов_И.О.lab06.tar"                               # сжатие
```

разархивировании каталоги могут оказаться моложе самого архива.

10 Программа-фильтр – такая программа, которая принимает входные данные на stdin и выводит в stdout (пример – gzip, wc).

Замечания относительно среды выполнения лабораторных и курсовых

Идеальной средой для выполнения практических, лабораторных и курсовых работ является Linux. Как вариант, можно использовать любой современный UNIX и даже MacOS[11] с соответствующими настройками. Возможно, последнему потребуются некоторые действия, чтобы ~~выбраться из тюрьмы~~ получить доступ к оболочке. Также придется установить gcc, libc и прочее с `www.gnu.org`.

Linux можно установить под Windows в виртуальной машине, однако наиболее удобным вариантом является dual-boot, поскольку некоторые курсовые невозможно выполнить в среде виртуальной машины.

Идеальным вариантом является вообще установка Windows в качестве гостевой ОС в Linux. Во-первых, это безопаснее, во-вторых спонтанное обновление Windows не снесет Linux из загрузки, в третьих, появляется беспрепятственная возможность обмена файлами между Windows и Linux, чего нельзя добиться, если хост — Windows, а Linux — гость.

Студенты спрашивают, а можно ли использовать WSL?

Можно, но не нужно. Добавляет студенту массу лишней работы. Как продвигается MS, WSL создана для «... использовать профессиональные инструменты программирования (Docker, Python, Git, компиляторы C++), которые изначально создавались для Linux, не покидая привычный интерфейс Windows». Задача же курса состоит, в том числе, для того, чтобы студент именно «покинул привычный интерфейс Windows» и освоил интерфейс оболочки, который на порядок более эффективен, в том числе и для «кухаркиных» работ, а также потому, что «привычный интерфейс Windows» препятствует освоению работы на уровне системного программирования, оперируя абстракциями, годными для освоения «кухарками», скрывая при этом суть реально происходящего.

Например, демонстрация лабораторной и/или курсовой должна выполняться под «натуральной» Linux/UNIX, в том числе и в отдельном терминале (Ctrl-Alt-Fn). Студент должен продемонстрировать, что он свободно себя чувствует вне «привычного интерфейса Windows», а поскольку WSL не изолирует студента от Windows, это препятствует приобретению навыков работы вне десктопа и смене типа мышления с пользовательского на системное и создает проблемы не только на лабораторных, но и в ходе сдачи экзамена.

Windows — не Linux, а Linux — не Windows. Сходство десктопов и графических интерфейсов не должно обманывать, операционная система — это не графические интерфейсы и тем более не десктоп, а набор системных вызовов, системных библиотек и утилит, обеспечивающих прикладным программам и прикладным библиотекам надлежащий доступ к аппаратной части. Простому пользователю этого знать не положено, а для системного программирования это составляет базу. Поэтому использование WSL не рекомендуется.

Какой Linux устанавливать?

Любой. Но лучше «rpm-based». Они стабильнее и безопаснее тех, что «debian-based». Бывалые и смелые могут использовать Arch.

Какой десктоп использовать?

На сегодня наиболее функциональным, достаточно легким и удобным является XFCE. Имеет огромные возможности по настройке «под себя».

Почему для оформления работ следует использовать LibreOffice?

Прежде всего потому, что msword отсутствует в Linux/UNIX. Но если бы и присутствовал, то использовать его не стоило бы — writer, в отличие от msword, использует стили оформления и не требует применения ручного форматирования. Отдельно стили страниц, отдельно стили аб-

11 MacOS — это UNIX.

зацев, стили перечислений, стили символов, и каждый можно настроить, привязать к комбинации клавиш, оформить в виде шаблона и использовать на постоянной основе. Такой подход существенно упрощает создание документов в строгом соответствии с требуемыми стандартами оформления, что серьезно облегчает подготовку простых документов, которые выглядят одинаково у всех, закрывая для любителей ~~ЖКП~~ ручного форматирования всякие «инклюзивные самовыражения». Как говорится, все кошки должны быть серы не только ночью, но и днем. Второе и главное преимущество — подготовка и редактирование больших и/или «долгоживущих» документов, в том числе и коллективная.

Практические занятия

2 ч. Заочное обучение.

Освоение эффективной работы в оболочке `bash` и файловом менеджере `mc`. Изучение наиболее часто используемых команд и утилит ОС.

Обеспечение: linux, стандартное информационное обеспечение (`info`, `man`).

Внешнее знакомство с POSIX-совместимой файловой системой — структура каталогов, жесткие и символические ссылки, права доступа, монтирование файловых систем, монтирование каталогов (`mount`, `mount –bind`).

Изучаемые вопросы:

- команды и утилиты: `man`, `info`, `mkdir`, `touch`, `rm`, `rmdir`, `cd`, `cat`, `sort`, `head`, `tail`, `tee`, `wc`, `chmod`, `ls`, `lsof`, `lsblk`, `lsusb`, `lscpu`, `ln`, `link`, `unlink`, `locale`, `iconv`, `kill`, `top`, `htop`, `ps`, `grep`, `diff`, `env`, `file`, `stat`, `find`, `tar`, `gzip`, `more`, `less`, `printf`, `time`, `...`;
- сцепление программ и соединение выходных и входных стандартных потоков;
- перенаправление вывода `stdout` и `stderr` в файлы;
- структура файловой системы, команды оболочки для работы с файлами и каталогами;
- архивирование файлов и каталогов.

Замечание

Использование оконных приложений и рабочего стола не допускается, только оболочка и mc.

Лабораторные

Лабораторная работа No 1. Знакомство с Linux/Unix и средой программирования. POSIX-совместимая файловая система.

Для дневного обучения только

Освоение эффективной работы в оболочке `bash` и файловом менеджере `mc`. Изучение наиболее часто используемых команд и утилит ОС.

Обеспечение: linux, стандартное информационное обеспечение (`info`, `man`).

Внешнее знакомство с POSIX-совместимой файловой системой — структура каталогов, жесткие и символические ссылки, права доступа, монтирование файловых систем, монтирование каталогов (`mount`, `mount –bind`).

Изучаемые вопросы:

- команды и утилиты: `man`, `info`, `mkdir`, `touch`, `rm`, `rmdir`, `cd`, `cat`, `sort`, `head`, `tail`, `tee`, `wc`, `chmod`, `ls`, `lsof`, `lsblk`, `lsusb`, `lscpu`, `ln`, `link`, `unlink`, `locale`, `iconv`, `kill`, `top`, `htop`, `ps`, `grep`, `diff`, `env`, `file`, `stat`, `find`, `tar`, `gzip`, `more`, `less`, `printf`, `time`, `...`;
- сцепление программ и соединение выходных и входных стандартных потоков;
- перенаправление вывода `stdout` и `stderr` в файлы;
- структура файловой системы, команды оболочки для работы с файлами и каталогами;
- архивирование файлов и каталогов.

Для дневного и заочного обучения

Изучаемые вопросы:

- структура файловой системы, содержимое `inode`, команды оболочки;
- знакомство с POSIX-совместимой файловой системой — `opendir(3)`, `readdir(3)`, `closedir(3)`, `stat(2)`, `lstat(2)`, `readlink(2)`, `realpath(3)`, `symlink(2)`, `link(2)`, `unlink(2)`, `...`

Задание

Разработать программу `dirwalk`, сканирующую файловую систему и выводящую в `stdout` информацию в соответствии с опциями программы.

Формат вывода аналогичен формату вывода утилиты `find`.

```
dirwalk [dir] [options]
```

`dir` — начальный каталог. Если опущен, текущий (`./`).

`options` — опции.

`-l` — только символические ссылки (`-type l`)

`-d` — только каталоги (`-type d`)

`-f` — только файлы (`-type f`)

`-s` — сортировать выход в соответствии с `LC_COLLATE`

Опции могут быть указаны как перед каталогом, так и после.

Опции могут быть указаны как раздельно, так и вместе (`-l -d`, `-ld`).

Если опции `ldf` опущены, выводятся и каталоги, файлы, и символические ссылки (симлинки).

Для обработки опций рекомендуется использовать `getopt(3)`.

Программа должна быть переносимой (возможности linux не используются, только POSIX).

Детали и требования к защите

Студент должен продемонстрировать понимание структуры файловой системы и уверенное владение утилитами, приведенными в условии лабораторной работы.

Для тестирования и демонстрации программы `dirwalk` преподавателю студент должен использовать системный каталог `/etc`, содержащий как минимум, три типа файлов — регулярные файлы, каталоги и симлинки.

С помощью утилит `find` и `wc` он должен получить четыре числа — общее количество файлов, количество регулярных файлов, каталогов и симлинков. Эти же четыре числа должны быть получены с помощью разработанной студентом программы `dirwalk`.

Для обхода дерева каталогов следует использовать системные вызовы, описанные в общей части руководства `man` (см. п. «Изучаемые вопросы»).

Использование оберток, системных вызовов и их возможностей, специфических для linux, не допускается.

Лабораторная работа No 2. Понятие процессов.

Изучение системных вызовов `fork()`, `execve()`, `getpid()`, `getppid()`, среды исполнения, функции `getenv()`, связи системного вызова `execve()` с функцией `main()` языка C.

Задание

Разработать две программы — `parent` и `child`.

Родительский процесс, используя `fork()`, запускает дочерний процесс и управляет его выполнением. Дочерний процесс, используя `execve()`) замещает код родительского процесса программой `child`, которая сканирует ввод с клавиатуры и выполняет указанные действия.

Родительский процесс, прежде чем запустить дочерний, создает сокращенную среду (окружение), которую будет использовать дочерний процесс при замене его кода программой с диска. Для этого студентом создается файл `env`, содержащий небольшой набор имен переменных окружения, передаваемых при вызове `execve()`.

Минимальный набор переменных в файле `env` должен включать `SHELL`, `HOME`, `HOSTNAME`, `LOGNAME`, `LANG`, `TERM`, `USER`, `LC_COLLATE`, `PATH`. Формат файла на усмотрение студента.

Перед запуском программы `parent` в ее окружении студентом создается переменная `CHILD_PATH` с именем каталога, где находится программа `child`. Если переменная `CHILD_PATH` не установлена, предполагается, что программа `child` располагается в текущем каталоге.

Родительский процесс (программа `parent`) после запуска получает переменные своего окружения и их значения, установленные оболочкой, сортирует в `LC_COLLATE=C` и выводит все в `stdout`. Затем читает файл `env` и формирует «сокращенную» среду для дочернего процесса в том виде, в котором она указывается в системном вызове `execve()`, используя значения для переменных из собственной среды. После этого ветвится и выполняет в цикле чтение `stdin` и выполняет по получению очередного символа следующие действия:

Символ «+»

Порождает дочерний процесс, используя `fork()`. Дочерний процесс, используя `execve(2)`, запускает очередной экземпляр программы `child`. Информацию о каталоге, где размещается `child`, `parent` получает из своего окружения, используя функцию `getenv()`.

Имя программы `child` (`argv[0]`) устанавливается как `child_XX`, где XX — порядковый номер от 00 до 99 (номер инкрементируется родителем).

Вторым параметром программы `child` является строка `"+"`.

Третьим параметром программы `child` является массив указателей на переменные «сокращенной» среды.

Дочерний процесс выводит свое имя, `pid` и `ppid` в `stdout`. Затем выводит свою среду в `stdout`, получая доступ к среде через внешнюю переменную `environ`.

Символ «*»

Дочерний процесс порождается и функционирует аналогично предыдущему случаю, однако получает доступ к среде через третий параметр функции `main()`.

Символ «q»

Завершает выполнение родительского процесса после завершения дочернего.

Требования к сборке

Программы компилируются с ключами

```
-W -Wall -Wextra -std=c11 -pedantic
```

Допускается использование ключей `-Wno-unused-parameter -Wno-unused-variable`.

Для компиляции, сборки и очистки используется `make`.

Лабораторная работа No 3. Взаимодействие и синхронизация процессов

Синхронизация процессов с помощью сигналов и обработка сигналов таймера.

Задание

Управление дочерними процессами и упорядочение вывода в `stdout` от них, используя сигналы `SIGUSR1` и `SIGUSR2`.

Действия родительского процесса

По нажатию клавиши «+» родительский процесс (P) порождает дочерний процесс (C_k) и сообщает об этом.

По нажатию клавиши «-» P удаляет последний порожденный C_k, сообщает об этом и о количестве оставшихся.

При вводе символа «`l`» выводится перечень родительских и дочерних процессов.

При вводе символа «`k`» P удаляет все C_k и сообщает об этом.

По нажатию клавиши «`q`» P удаляет все `C_k`, сообщает об этом и завершается.

Действия дочернего процесса

Дочерний процесс во внешнем цикле заводит будильник (`nanosleep(2)`) и входит в вечный цикл, в котором заполняет структуру, содержащую пару переменных типа `int`, значениями {0, 0} и {1, 1} в режиме чередования. Поскольку заполнение не атомарно, в момент срабатывания будильника в структуре может оказаться любая возможная комбинация из 0 и 1.

При получении сигнала от будильника проверяет содержимое структуры, собирает статистику и повторяет тело внешнего цикла.

Через заданное количество повторений внешнего цикла (например, через 101) дочерний процесс выводит свои PPID, PID и 4 числа — количество разных пар, зарегистрированных в момент получения сигнала от будильника.

Вывод осуществляется в одну строку и должен быть отформатирован.

Следует подобрать интервал времени ожидания и количество повторений внешнего цикла, чтобы статистика была значимой (101, как вариант).

Сообщения выводятся в `stdout`.

Сообщения процессов должны содержать идентифицирующие их данные, чтобы можно было фильтровать вывод утилитой `grep`.

Требования к сборке аналогичны требованиям из лабораторной № 2.

Лабораторная работа No 4. Задача производители-потребители для процессов

Основной процесс создает очередь сообщений, после чего ожидает и обрабатывает нажатия клавиш, порождая и завершая процессы двух типов —производители и потребители.

Очередь сообщений представляет собой классическую структуру — кольцевой буфер, содержащий указатели на сообщения, и пара указателей на голову и хвост. Помимо этого очередь содержит счетчик добавленных сообщений, счетчик извлеченных, количество сообщений в очереди и количество свободного места в очереди.

Производители формируют сообщения и, если в очереди есть место, перемещают их туда.

Потребители, если в очереди есть сообщения, извлекают их оттуда, обрабатывают и освобождают память с ними связанную.

Для работы используются два семафора для заполнения и извлечения, а также мьютекс или одноместный семафор для монопольного доступа к очереди.

Сообщения имеют следующий формат (размер и смещение в байтах):

Имя	**Размер**	**Смещение**	**Описание**
`type`	1	0	тип сообщения
`hash`	2	1	контрольные данные
`size`	1	3	длина данных в байтах (от 0 до 255)
`data`	`((size + 4)/4)*4`	4	данные сообщения

Производители генерируют сообщения, используя системный генератор случайных чисел `rand(3)` или `rand_r(3)` для `size` и `data`. В качестве результата для `size` используется остаток от деления на 256. Реальный размер сообщения на единицу больше и лежит в интервале (1, 256).

Поле `data` имеет длину, кратную 4-м байтам.

При формировании сообщения контрольные данные формируются только из байт сообщения длиной `size + 1`.

Значение поля `hash` при вычислении контрольных данных принимается равным нулю.

Для расчета контрольных данных можно использовать любой подходящий алгоритм на выбор студента.

В качестве семафоров используются семафоры System V.

После помещения значения в очередь перед освобождением мьютекса очереди производитель инкрементирует счетчик добавленных сообщений. Затем после освобождения мьютекса выводит строку на `stdout`, содержащую помимо всего новое значение этого счетчика.

Потребитель, получив доступ к очереди, извлекает сообщение и удаляет его из очереди. Перед освобождением мьютекса очереди инкрементирует счетчик извлеченных сообщений. Затем после освобождения мьютекса проверяет контрольные данные и выводит строку на `stdout`, содержащую помимо всего новое значение счетчика извлеченных сообщений.

При получении сигнала о завершении процесс должен завершить свой цикл и только после этого завершиться, не входя в новый.

Следует предусмотреть задержки, чтобы вывод можно было успеть прочитать в процессе работы программы.

Следует предусмотреть защиту от тупиковых ситуаций из-за отсутствия производителей или потребителей.

Следует предусмотреть нажатие клавиши для просмотра состояния (размер очереди, сколько занято и сколько свободно, столько производителей и сколько потребителей).

Требования к сборке аналогичны требованиям из лабораторной № 2.

Лабораторная работа No 5. Потоки исполнения, взаимодействие и синхронизация

Здесь две задачи. Обе «производители-потребители» для потоков.

Изучаемые системные вызовы: `pthread_create()`, `pthread_exit()`, `pthread_join()`, `pthread_yield()`, `pthread_cancel()`, `pthread_cond_init()`, `pthread_cond_destroy()`, `pthread_cond_*wait()`, `pthread_cond_signal()`.

Две лабораторных в одной:

5.1) Аналогична лабораторной No 4, но только с потоками, posix-семафорами и мьютексом в рамках одного процесса.

Дополнительно обрабатывается еще две клавиши – увеличение и уменьшение размера очереди. Следует предусмотреть обработку запроса на уменьшение очереди таким образом, чтобы при появлении пустого места уменьшался размер очереди, а не очередной производитель размещал там свое сообщение.

5.2 Аналогична лабораторной № 5.1, но с использованием условных переменных (см. лекции СПОВМ/ОСиСП).

Требования к сборке аналогичны требованиям из лабораторной № 2.

Лабораторная работа No 6. Работа с файлами, отображенными в память

Кооперация потоков для высокопроизводительной обработки больших файлов.

Изучаемые системные вызовы: `pthread_barrier_init()`, `pthread_barrier_destroy()`, `pthread_barrier_wait()`, `mmap()`, `munmap()`.

Задание

Написать многопоточную программу `sort_index` для сортировки вторичного индексного файла таблицы базы данных, работающую с файлом с использованием отображение файлов в адресное пространство процесса. Программа должна запускаться следующим образом:

```
$ sort_index memsize blocks threads filename
```

Параметры командной строки:

`memsize` — размер рабочего буфера, кратный размеру страницы (`getpagesize(2)`);

`blocks` — порядок разбиения буфера (количество блоков);

`threads` — количество потоков (от `k` до `N`), где `k` — количество процессорных ядер, `N` — максимальное количество потоков (k <= N <= 8k).

`filename` — имя файла

Количество блоков должно быть степенью двойки и превышать количество потоков не менее, чем в 4 раза (при k = 4 количество блоков не должно быть менее 16). Соответственно, размер файла должен удовлетворять указанным ограничениям.

Для целей тестирования следует написать программу `gen` , которая будет генерировать неотсортированный индексный файл, и программу `view` для отображения индексного файла на `stdout`.

Алгоритм программы генерации

Генерируемый файл представляет собой вторичный индекс по времени и состоит из заголовка и индексных записей фиксированной длины.

Индексная запись имеет следующую структуру:

```
struct index_s {
    double   time_mark; // временная метка (модифицированная юлианская дата)
    uint64_t recno;     // номер записи в таблице БД (первичный индекс)
};
```

Заголовок представляет собой следующую структуру

```
struct index_hdr_s {
    uint64_t       recsords; // количество записей
    struct index_s idx[];    // массив записей в количестве records
};
```

Временная метка определяется в модифицированный юлианских днях. Целая часть лежит в пределах от 15020.0 (1900.01.01-0:0:0.0) до «вчера»[12]. Дробная – это часть дня. Для генерации целой и дробной частей временной метки следует использовать системный генератор случайных чисел `rand(3)` или `rand_r(3)`.

Первичный индекс, как вариант, может заполняться последовательно, начиная с 1, но может быть случайным целым > 0 (в программе сортировки не используется).

12 https://en.wikipedia.org/wiki/Julian_day. Находим в таблице вариантов «Modified JD» и получаем значение даты на сегодня. вычитаем единицу и целую часть используем как максимальное значение целой части генерируемой даты.

Размер индекса в записях должен быть кратен 256 и кратно превышать планируемую выделенную память для отображения. Размер индекса и имя файла указывается при запуске программы генерации.

Алгоритм программы сортировки

1) Основной поток запускает `threads` потоков, сообщая им адрес буфера, размер блока `memsize/blocks`, и их номер от 1 до `threads - 1`, используя возможность передачи аргумента для `start_routine`. Порожденные потоки останавливаются на барьере, ожидая прихода основного.

2) Основной поток с номером 0 открывает файл, отображает его часть размером `memsize` на память и синхронизируется на барьере. Барьер «открывается» и все `threads` потоков входят на равных в фазу сортировки.

3) Фаза сортировки

С каждым из блоков связана карта (массив) отсортированных блоков, в которой изначально блоки с 0 по `threads-1` отмечены, как занятые.

Поток `n` начинает с того, что выбирает из массива блок со своим номером и его сортирует, используя `qsort(3)`. После того, как поток отсортировал свой первый блок, он на основе конкурентного захвата мьютекса, связанного с картой, получает к ней эксклюзивный доступ, отмечает следующий свободный блок, как занятый, освобождает мьютекс и приступает к его сортировке.

Если свободных блоков нет, синхронизируется на барьере. После прохождения барьера все блоки будут отсортированы.

4) Фаза слияния

Поскольку блоков степень двойки, слияния производятся парами в цикле.

Поток 0 сливает блоки 0 и1, поток 1 — блоки 2 и 3, и так далее.

Для отметки слитых пар и не слитых используется половина карты. Если для потока нет пары слияния, он синхронизируется на барьере.

В результате слияния количество блоков, подлежащих слиянию сокращается в два раза, а размер их в два раза увеличивается.

После очередного прохождения барьера количество блоков, подлежащих слиянию, станет меньше количества потоков. В этом случае распределение блоков между потоками осуществляется на основе конкурентного захвата мьютекса, связанного с картой. Потоки, которым не досталось блока, синхронизируются на барьере.

Когда осталась последняя пара, все потоки с номером не равным нулю синхронизируются на барьере, о поток с номером 0 выполняет слияние последней пары.

После слияния буфер становится отсортирован и подлежит сбросу в файл (`munmap()`).

Если не весь файл обработан, продолжаем с шага 2).

Если весь файл обработан, основной поток отправляет запрос отмены порожденным потокам, выполняет слияние отсортированных частей файла и завершается.

Как вариант, потоки, которым не досталось блоков для слияния, завершаются.

Лабораторная работа No 7. Блокировки чтения/записи

Конкурентный доступ к совместно используемому файлу, используя блокировку чтения-записи. Изучаемые системные вызовы: `fcntl(F_GETLK, F_SETLK, F_SETLKW, F_UNLK)`.

Программа в режиме конкурентного доступа читает из и пишет в файл, содержащий записи фиксированного формата. Формат записей произвольный. Примерный формат записи:

```
struct record_s {
    char     name[80];    // Ф.И.О. студента
    char     address[80]; // адрес проживания
    uint8_t  semester;    // семестр
};
```

Файл должен содержать не менее 10 записей. Создается и наполняется с помощью любых средств.

Программа должна выполнять следующие операции:

1) `LST` – Отображение содержимого файла с последовательной нумерацией записей
2) `GET Rec_No` – получение записи с порядковым номером `Rec_No`;
3) Модификацию полей записи
4) `PUT` – сохранение последней прочитанной и модифицированной записи по месту.

Интерфейс с пользователем на «вкус» студента.

Алгоритм конкурентного доступ к записи

```
    REC <-- get(Rec_No)           // читаем запись с номером Rec_No в буфер REC
Again:
    REC_WRK <-- REC               // копируем в рабочую область

    ... // делаем что-нибудь с записью в REC_WRK (модификация) и желаем ее сохранить

    if (REC_WRK != REC) { //
        lock(Rec_No)              // блокируем запись для модификации в файле
        REC_NEW <-- get(Rec_No)   // и перечитываем запись из файла
        if (REC_NEW != REC) {     // кто-то изменил запись после получения ее нами
            unlock(Rec_No)        // освобождаем запись
            message               // сообщаем, что запись кто-то изменил
            REC <--  REC_NEW      // повторим все с ее новым содержимым если нужно
            goto Again
        }
        put(REC_WRK, Rec_No)      // сохраняем новое содержимое
        unlock(Rec_No)            // освобождаем запись
    }
```

Для отладки и тестирования используется не менее двух экземпляров программы.

Для блокирования записей используется `fcntl()` и исключительные блокировки на основе файловых дескрипторов (`F_OFD_*`).

Для чтения записи следует использовать совместную блокировку, защищающую чтение от записи другими процессами.

Лабораторная работа No 8. Сокеты. Взаимодействие процессов.

Задача – разработка многопоточного сервера и клиента, работающих по простому протоколу.

Изучаемые системные вызовы: `socket()`, `bind()`, `listen()`, `conect()`, `accept()` и прочих, связанных с адресацией в домене AF_INET.

Протокол должен содержать следующие запросы:

`ECHO` – эхо-запрос, возвращающий без изменений полученное от клиента;

`QUIT` – запрос на завершение сеанса;

`INFO` – запрос на получения общей информации о сервере;

`CD` – изменить текущий каталог на сервере;

`LIST` – вернуть список файловых объектов из текущего каталога.

Протокол может содержать дополнительные запросы по выбору студента, не выходящие за пределы корневого каталога сервера и не изменяющих файловую систему в его дереве.

Запросы клиенту отправляются на `stdin`.

Ответы сервера и ошибки протокола выводятся на `stdout`.

Ошибки системы выводятся на `stderr`.

Подсказка клиента для ввода запросов – символ `'>'`.

Сервер принимает корневой каталог сервера, с которым он будет работать, и номер порта, на котором будет слушать, и выводит протокол работы в `stdout`.

```
$ myserver root_dir port_no
Готов.
```

Формат протокола произвольный, каждое событие занимает одну строку, первое поле – дата и время в формате YYYY.MM.DD-hh:mm:ss.sss).

Клиент помимо интерактивных запросов принимает запросы из файла. Файл с запросами указывается с использованием префикса `'@'`:

```
$ myclient server.domen
Вас приветсвует учебный сервер 'myserver'
> @file
> ECHO какой-то_текст
какой-то_текст
> LIST
dir1
dir2
file
> CD dir1
dir1> QUIT
BYE
$
```

ECHO – эхо-запрос, возвращающий без изменений полученное от клиента.

```
> ECHO "произвольный текст"
произвольный текст
>
```

QUIT – запрос на завершение сеанса

```
> QUIT
BYE
$
```

INFO – запрос на получения общей информации о сервере.
Сервер отправляет текстовый файл с соответствующей информацией.

```
> INFO
Вас приветсвует учебный сервер 'myserver'
>
```

Этот же файл сервер отправляет клиенту при установлении сеанса.

LIST – вернуть список файловых объектов из текущего каталога.
Текущий каталог — каталог в дереве каталогов сервера. Корневой каталог сервера устанавливается из командной строки при старте сервера.

```
> LIST
dir1/
dir2/
file1
file2 --> dir2/file2
file3 -->> dir1/file
>
```

Каталоги выводятся с суффиксом `'/'` после имени, файлы — как есть, симлинки на регулярные файлы разрешаются через `'-->'`, симлинки на симлинки разрешаются через `'-->>'`. Корневой каталог сервера при выводе указывается префиксом `'/'` перед именем.

CD – изменить текущий каталог на сервере
Выход за пределы дерева корневого каталога сервера запрещается, команда безмолвно игнорируется

```
> CD dir2
dir2> LIST
file2
dir2> CD ../dir1
dir1> LIST
file --> /file1
dir1> CD ..
> CD ..
>
```

Соединения функционируют независимо, т.е. текущий каталог у каждого соединения свой.

Примечания:

Раскрашивать вывод не нужно.
Для разработки и отладки лабораторной следует использовать редактор или IDE, поддерживающие несколько запущенных экземпляров, каждый со своей конфигурацией, и поддерживающие отладку прямо в окне с кодом, например, slickedit (лучший выбор).

Приложение А. Структура проекта и примеры `makefile`

Один исходный файл на С

```
# makefile
CC = gcc
CFLAGS = -W -Wall -Wextra -std=c11
.PHONY: clean

all: prog
prog: prog.c makefile
        $(CC) $(CFLAGS) prog.c -o prog
clean:
        rm prog
```

Компиляция и сборка

```
$ make
gcc -W -Wall -Wextra -std=c11 prog.c -o prog
$
```

Запуск

```
$ ./prog
```

Библиотека и программа, ее использующая

Структура проекта для сборки библиотеки и программы для ее отладки и тестирования[13]:

```
$ tree -a
.
├── .git            -- каталог локального git-репозитория
:                   -- содержимое репозитория
├── Makefile        -- файл, управляющий сборкой
├── build           -- каталог сборки
│   ├── .gitignore  -- файл неотслеживания git
│   ├── debug       -- каталог варианта отладки
│   │   ├── lib.o   -- объектный модуль библиотеки
│   │   ├── main.o  -- объектный модуль программы
│   │   └── test    -- исполняемый модуль программы отладки и тестирования
│   └── release     -- каталог варианта выпуска
│   │   ├── lib.o   -- объектный модуль библиотеки
│   │   ├── main.o  -- объектный модуль программы
│   │   └── test    -- исполняемый модуль программы отладки и тестирования
└── src             -- каталог с исходными кодами
    ├── lib.c       -- код библиотеки
    ├── lib.h       -- файл заголовка библиотеки
    └── main.c      -- код тестовой программы
```

Содержимое `Makefile`

```
$ cat Makefile
#makefile
CC = gcc
CFLAGS = -std=c11 -g2 -ggdb -pedantic -W -Wall -Wextra
```

13 ftp://lsi.bas-net.by/ОсиСП-2023/code/makefile/lib_ex/

```
.SUFFIXES:
.SUFFIXES: .c .o

DEBUG   = ./build/linux/debug
RELEASE = ./build/linux/release
OUT_DIR = $(DEBUG)
vpath %.c src
vpath %.h src
vpath %.o build/linux/debug

ifeq ($(MODE), release)
  CFLAGS = -std=c11 -pedantic -W -Wall -Wextra -Werror
  OUT_DIR = $(RELEASE)
  vpath %.o build/linux/release
endif

objects =  $(OUT_DIR)/main.o $(OUT_DIR)/lib.o
#objects =  main.o lib.o

prog = $(OUT_DIR)/test

all: $(prog)

$(prog) : $(objects)
    $(CC) $(CFLAGS) $(objects) -o $@

$(OUT_DIR)/%.o : %.c
    $(CC) -c $(CFLAGS) $^ -o $@

.PHONY: clean
clean:
    @rm -rf $(DEBUG)/* $(RELEASE)/* test
```

Один исходный файл на ассемблере

```
NAME=hello
INCLUDES=stud_io.inc

AS=nasm
CC=gcc
LD=ld # или gcc

LDFLAGS=-m elf_i386           # -m32 для LD=gcc
ASFLAGS=-Wall -f elf -g

.SUFFIXES:
.SUFFIXES: .o .c .asm

all: $(NAME)

$(NAME): $(NAME).o
    $(LD) $(LDFLAGS) $^ -o $@

.PHONY: clean
clean:
    $(RM) $(NAME) *.o *.lst

$(NAME).o: $(NAME).asm $(INCLUDES) makefile
    $(AS) $(ASFLAGS) -l $(*F).lst $< -o $@
```

Приложение Б. Требования к оформлению исходных файлов

1) Функция `main()` должна быть первой в исходном модуле, который ее содержит.

2) При запуске программы, требующей параметры, без параметров программа должна выводить подсказку о ее использовании.

3) В именах идентификаторов, типов, структур, перечислений используются только символы нижнего регистра и символ подчеркивания '_'. Образцом именования идентификаторов является стандарт «ISO/IEC 9899–2011 Programming Language — C». Использование горбатого стиля запрещается.

4) Имена макросов именуются в верхнем регистре.

5) Имена тегов структур имеют суффикс «`_s`» (`struct name_s {}`)

6) Имена типов, созданных с помощью `typedef` имеют суффикс «`_t`».

7) В начале исходного модуля должен располагаться комментарий с описанием.

8) Всем определениям функций должны предшествовать комментарии, описывающие назначение, что принимает и что возвращает. Эта же информация должна присутствовать в разделе описания функциональной структуры проекта.

9) Комментарии выполняются на русском языке в кодировке utf8 или английском, если студент хорошо владеет языком и может без позора письменно выражать мысли.

10) Условная компиляция

В случае, если параметры конфигурации при создании программы уже известны, предпочтительно использовать `if (... )` вместо условной компиляции, поскольку в первом случае компилятор может выполнить более тщательную проверку всех возможных ветвей кода.

Например, стоит писать

```
if (HAS_FOO)
  ...
else
  ...
```

вместо

```
#ifdef HAS_FOO
  ...
#else
  ...
#endif
```

Современный компилятор, такой как GCC, в обоих случаях генерирует один и тот же код. Вышеуказанный метод предполагает, что `HAS_FOO` определяется как 0 или 1.

11) Поведение программ для всех программ

Стандарт языка C гласит, что почти все расширения C запрещены. В лабораторных и курсовых работах мы следуем стандарту, в связи с чем указываем при компиляции «`--pedantic`» — в этом случае GCC будет вести себя, как 100% реализация стандарта.

Следует избегать произвольных ограничений на длину или количество любых структур данных, включая имена файлов, строки, файлы и символы, используя массивы. Вместо этого следует использовать динамическое выделение памяти.

Программы должны правильно работать с многобайтовыми кодировками символов, такими как UTF-8. Для работы с различными кодировками можно использовать `libiconv`.

В обязательном порядке следует проверять каждый системный вызов на предмет возврата ошибки, если не предполагается, что ошибки должны игнорироваться специально.

Следует проверять каждый вызов `malloc()` или `realloc()`, чтобы увидеть, вернул ли он `NULL`.

Следует проверять realloc() даже если размер блока уменьшается, поскольку в системе, которая округляет размеры блоков до степени 2, realloc() при запросе уменьшить выделенное место, с высокой вероятностью получит другой блок.

Следует ожидать, что free() изменит содержимое освобожденного блока. Все, что необходимо получить из блока, должно быть получено перед вызовом free().

Если malloc() завершается сбоем в неинтерактивной программе, это должно быть фатальной ошибкой (abort()).

В интерактивной программе (той, которая считывает команды от пользователя) следует прервать выполнение команды и вернуться к циклу чтения команд. Это позволяет пользователю отключить другие процессы, чтобы освободить виртуальную память, а затем повторить попытку выполнения команды.

Если в статическую память во время выполнения программы должна быть осуществляться запись, следует использовать явный код C для ее инициализации. Таким образом, перезапуск программы без ее перезагрузки или ее части приведет к повторной инициализации этих переменных.

При создании временных файлов следует проверять переменную среды TMPDIR — если эта переменная определена, следует использовать указанный каталог вместо /tmp.

Рекомендуется следовать рекомендациям POSIX для параметров командной строки программы. Самый простой способ это сделать — использовать getopt для их анализа.

При завершении процесса или потока следует освободить все полученные ресурсы. Это в первую очередь относится к ресурсам, полученным при использовании вызовов pthreads.

Файлы, имена которых передаются в качестве обычных аргументов должны быть только входными.

Выходные файлы должны указываться с использованием опций (предпочтительно «-o» или «--output»).

Стиль отступов должен быть аналогичен стилю, который используется в стандарте, который можно найти на ftp://lsi.bas-net.by.

Приложение В. Требования к оформлению лабораторной работы

Отчет о лабораторной работе предоставляется в формате документов `pdf` или в виде твердой копии и должен содержать следующие элементы:

- титульный лист;
- основную часть.

Титульный лист

Титульный лист должен содержать:

- наименование курса (Операционные системы и системное программирование);
- номер и название лабораторной работы;
- номер группы, фамилию и инициалы студента.

Основная часть

Основная часть состоит из:

- раздела с условиями лабораторной работы (может копироваться из настоящего документа);
- раздела с описанием алгоритмов и решений;
- раздела с описанием функциональной структуры проекта лабораторной работы;
- раздела с описанием порядка сборки и использования;
- раздела с описанием метода тестирования и результатами тестирования;

Раздел с аналогами, как того требует дипломная работа, может быть опущен.

1) Текстовая часть отчета выполняется в один интервал пропорциональным шрифтом с засечками (serif) размером 12-14 pt.

Абзац начинается с красной строки размером 5 букв 'н'.

Следует различать короткое тире (U+2013 m-dash), длинное тире (U+2014 n-dash) и дефис (U+002D) и не заменять одно другим.

2) Программный код, отдельные его элементы по тексту, команды и представленное в отчете содержимое файлов проекта выполняется моноширинным шрифтом (`typewriter`) визуально того же размера и одинаковой плотности с плотностью основного текста.

Шрифт должен быть таким, у которого глиф «латинская малая буква L» (U+006C) визуально отличается от цифры «один» (U+0031), например, «`l` и 1» А цифра «ноль» (U+0031) — от буквы O. Применение шрифтов семейства «Courier» не допускается.

3) Надписи на диаграммах и рисунках, если таковые имеются, выполняются пропорциональным шрифтом без засечек (sans). Размер шрифта надписей не должен быть больше размера основного текста, но и не менее 8 pt.

4) Перечисления оформляются в соответствии с требованием ГОСТ 2.105 «Общие требования к текстовым документам».

5) Скриншоты в общем случае не допускаются — запуск команд, программ и их вывод в консоль копируются в буфер обмена и вставляются в текст отчета черным по белому в соответствии с 2).

6) Диаграммы, схемы и прочий графический материал, если есть в нем необходимость, выполняется студентом полностью самостоятельно — копипаста с чужих ресурсов категорически запрещена.

Вышеуказанный графический материал выполняется в виде отдельного файла, вставляется в записку в векторном или растровом формате и располагается в дереве проекта лабораторной. Рисовать по тексту записки запрещается. Векторный формат графического материала должен поддерживаться во всех ОС. Растровый формат вставляемого изображения должен иметь разрешение не менее 600 dpi.

7) Отчет исполняется на русском языке с использованием его правил (`gramota.ru`) без изъятия. Это касается также использования знаков препинания, дефиса, короткого и длинного тире.

Приложение Г. Требования к почтовому сообщению

Требования к почтовому сообщению, содержащему отчетные материалы, устанавливает преподаватель если он использует данный способ взаимодействия. Тем не менее:

1) Сообщения с вложениями лабораторных и курсовых должны быть только оригинальными, т.е. не содержащими строки `In-Reply-To:`.

Отправка ответов (`In-Reply-To:`) допускается только на ответ преподавателя.

2) Наличие темы сообщения обязательно. Все сообщения без темы сразу же отправляются в `dev/null` и карма студента понижается до 4 (один раз и сразу за саботаж правил почтового взаимодействия). Тема сообщения должна содержать следующую информацию в следующем порядке:

а) номер группы;

б) фамилия с инициалами;

в) обязательный текст, указывающий тип содержимого;

г) необязательный произвольный текст.

Все элементы отделяются друг от друга одним пробелом (␣ — это символ пробела для твердых копий, если необходимо показать, что данное поле кодом 20h).

Примеры:

`123456␣Иванов␣А.Б.␣Лабораторная␣01`

`123456␣Иванов␣А.Б.␣Курсовой␣проект`

3) Вложение должно представлять собой один файл-архив (см. п. 14 Общих замечаний) и содержать только одну лабораторную или одну версию курсового проекта. Отправка нескольких файлов или нескольких лабораторных в одном файле не допускается. несколько файлов вложений в одном сообщении не допускается. Имя файла должно состоять из пунктов а), б) и в), разделенных подчеркиванием '_' вместо пробелов. Имена всех отправляемых студентом файлов с лабораторными должны различаться только номером лабораторной. Файлы с курсовой работой должны иметь одно и то же имя.

4) Если почтовый сервер, используемый для доставки почты, не может отправить сообщение по причине превышения размера сообщения, что может иметь место в ряде случаев для курсового проекта, допускается расчленение проекта на несколько вложений, которые отправляются отдельными сообщениями.

Для расчленения следует использовать утилиту `split`.

Для слияния компонент следует использовать утилиту `cat`.